TRƯỜNG ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Tetris Game

Group 4

1. Đoàn Đức Anh - 202417191
2. Bùi Đức Sơn - 202417193
3. Phạm Lê Công - 202417102
4. Phạm Đức Gia Bảo - 202417101
5. Nguyễn Văn Tùng Cương - 202417103

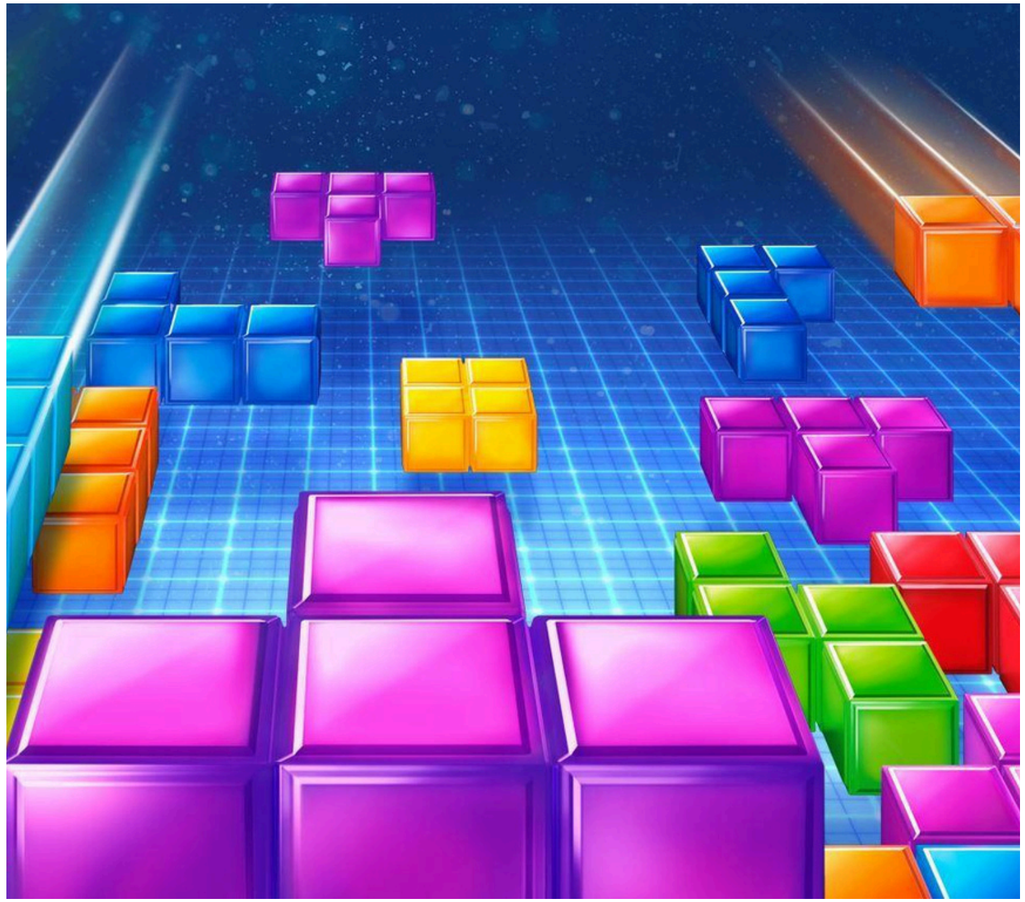
ONE LOVE. ONE FUTURE.

1. Member and assignment

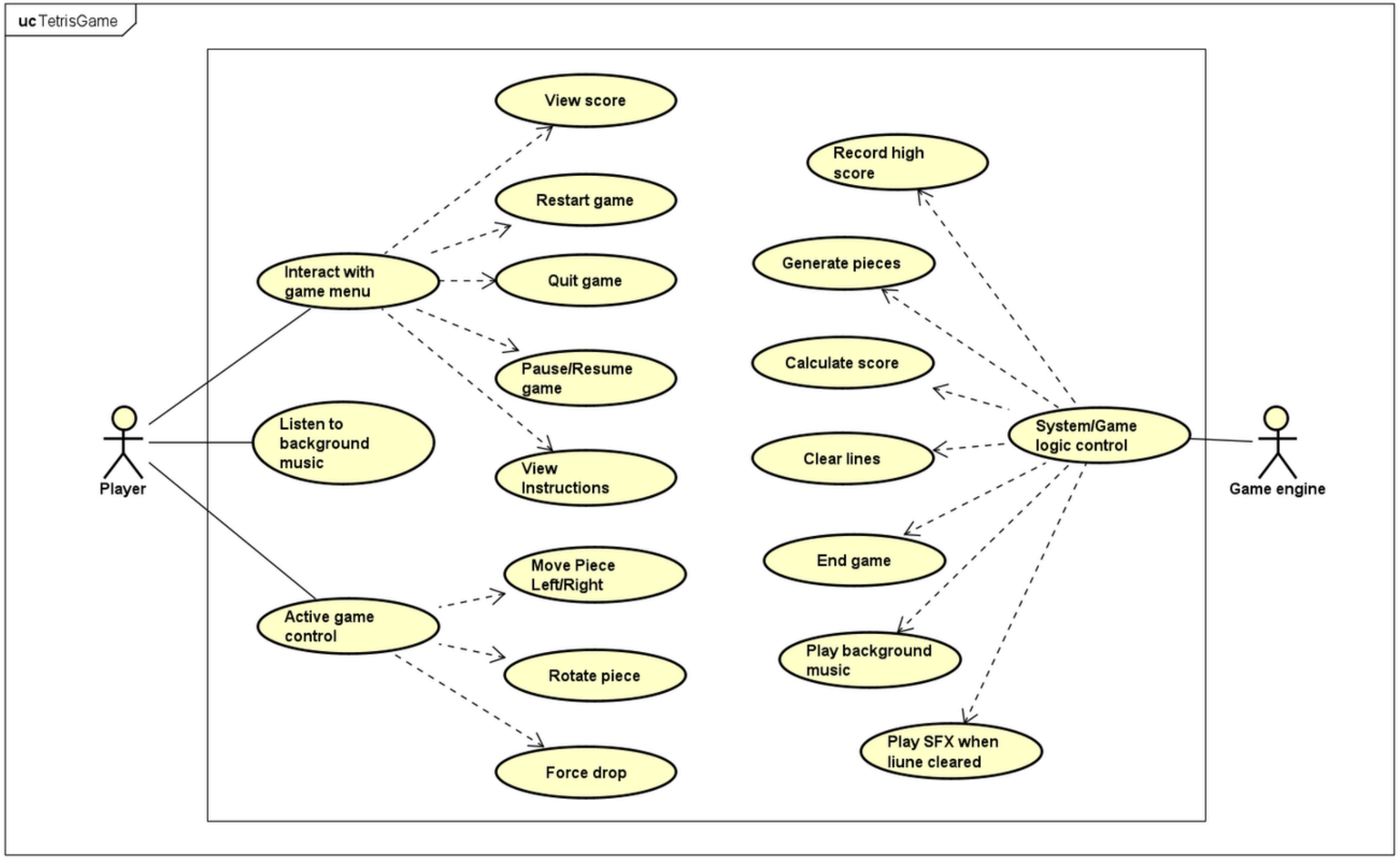
Group members	Student ID	Class/methods assigned	Percentage (%)
Đoàn Đức Anh	202417091	Classes: Cell, Board, Tetromino, StraightShape, SquareShape, IShape, SShape, ZShape, TShape,JShape, Main	20
Bùi Đức Sơn	202417193	Classes: MenuPanel, HelpPanel, SoundController, GamePanel(modify: bgmToggleButton), DifficultyPanel (modify: difficultyComboBox, paintComponent)	20
Phạm Lê Công	202417102	Classes: MainFrame, GamePanel(modify: drawNextPieces, drawScore), GameEngine(modify: spawnNewPiece, createNextPiece, addPiecetoQueue, getNewPiece, getUpcomingPieces	20
Phạm Đức Gia Bảo	202417101	Classes: GamePanel, DifficultyPanel, Constants, GameEngine(modify: scoreMultiplier, calculateScore, setDifficultyDelay), MainFrame(modify: constructor), HelpPanel(modify: constructor)	20
Nguyễn Văn Tùng Cương	202417103	Classes: GameEngine, GamePanel(modify: setEngineAndRouting, setupKeyBindings, paintComponent, drawSingleBlock, drawNextPieces, drawGrid, drawLockedBlocks, drawActivePiece, drawShadowPiece, drawScore, setActivePiece, refreshBoard)	20

2. Problem statement

Tetris is a classic puzzle game where players arrange falling blocks to clear horizontal lines. This project applies object-oriented programming (OOP) to build a structured, maintainable, and extensible system.



3. Use case diagram

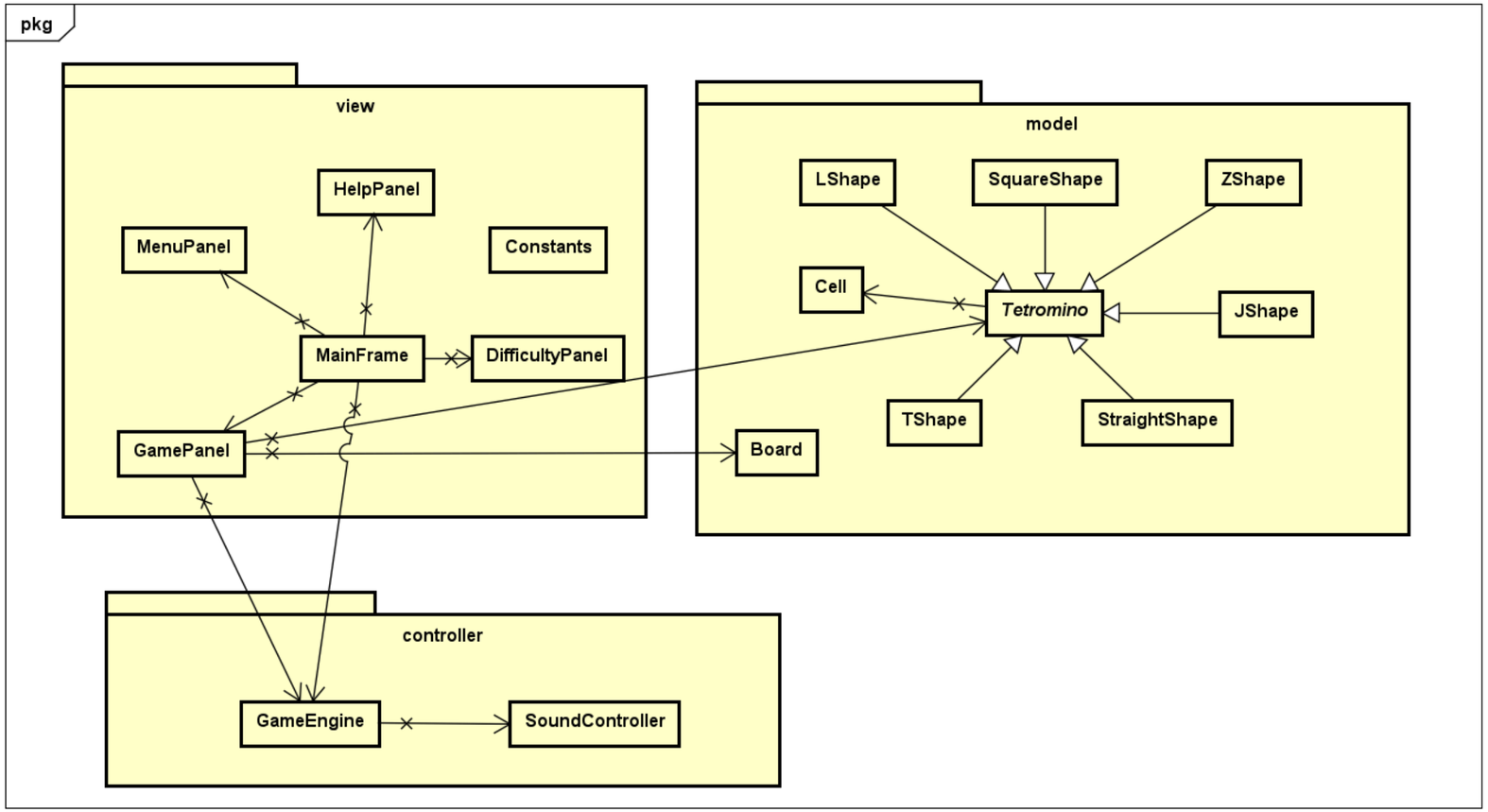


The Player navigates menus and moves, The automated Game Engine handles background tasks

4. General class diagram

MVC architecture

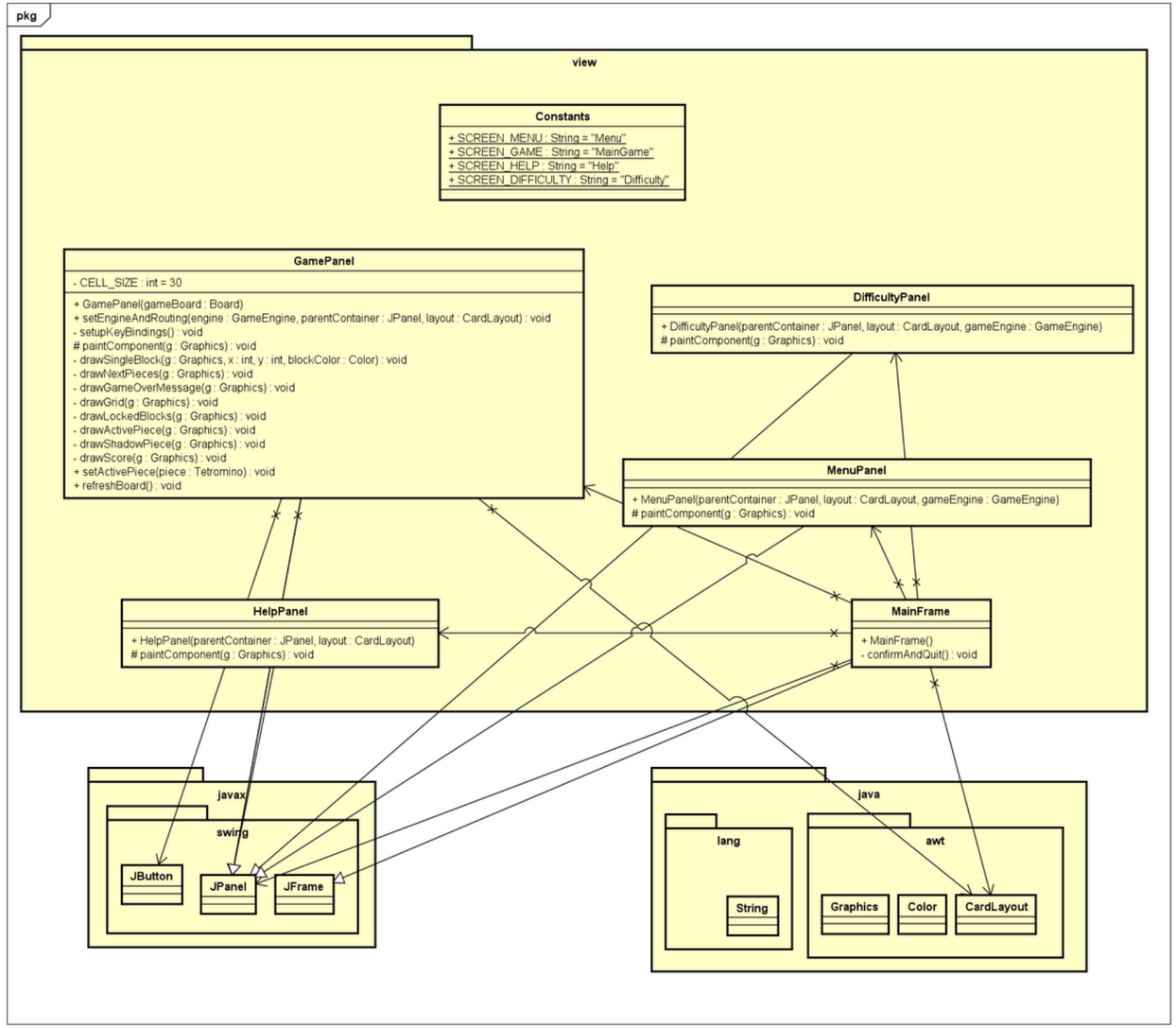
- The Model handles game data like the grid board and tetris shapes,
- The View manages visual panels like menus and screens.
- The Controller runs the game engine logic to connect and coordinate the data with the visuals.



5. View package class diagram

Internal structure of the game's View package.

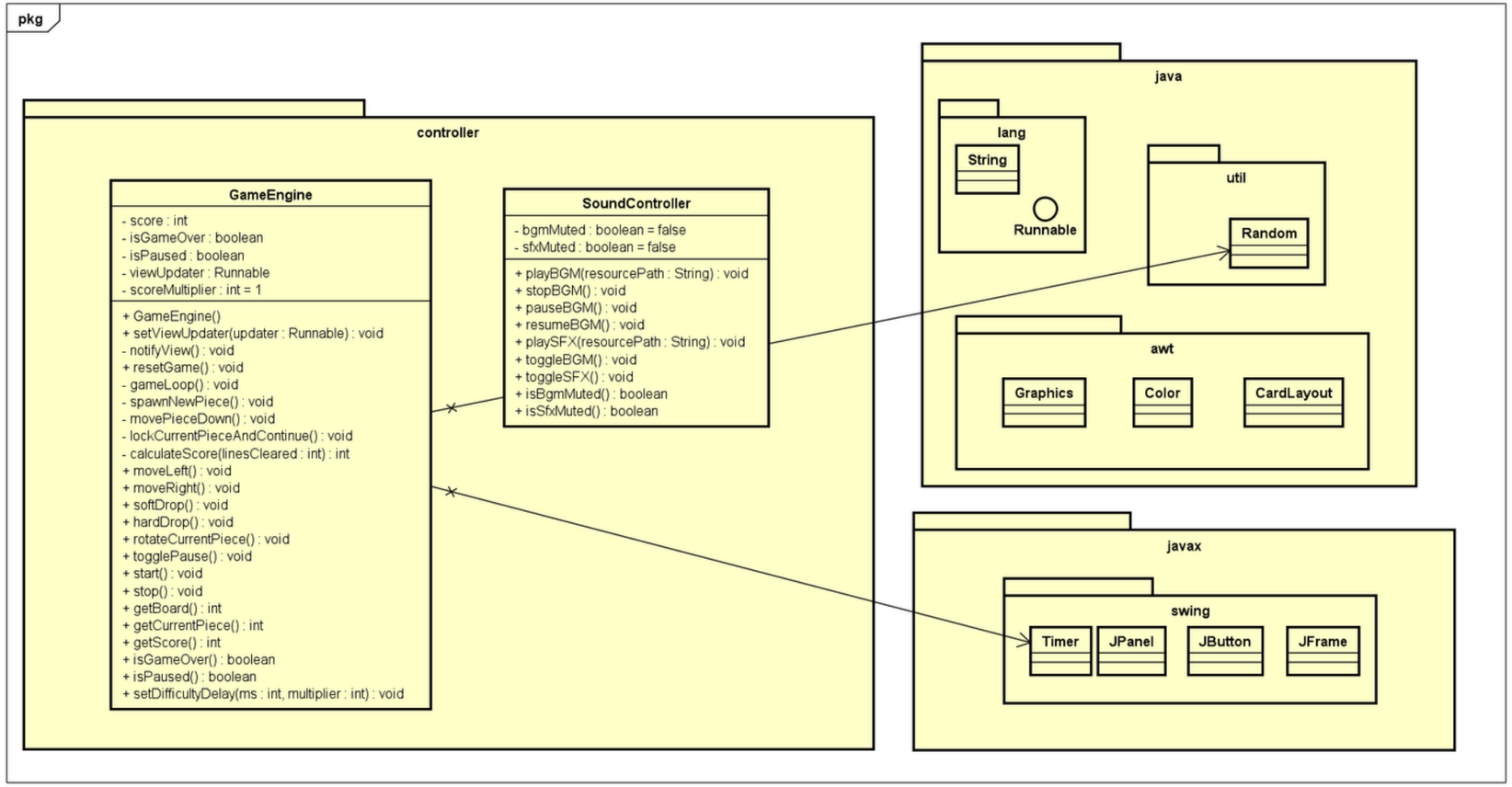
- Defines separate visual panels for the menu, difficulty settings, help screen, and core gameplay board.
- Utilize Java Swing and AWT to handle window layouts, colors, and graphics.



6. Controller package class diagram

Internal structure of the game's Controller package

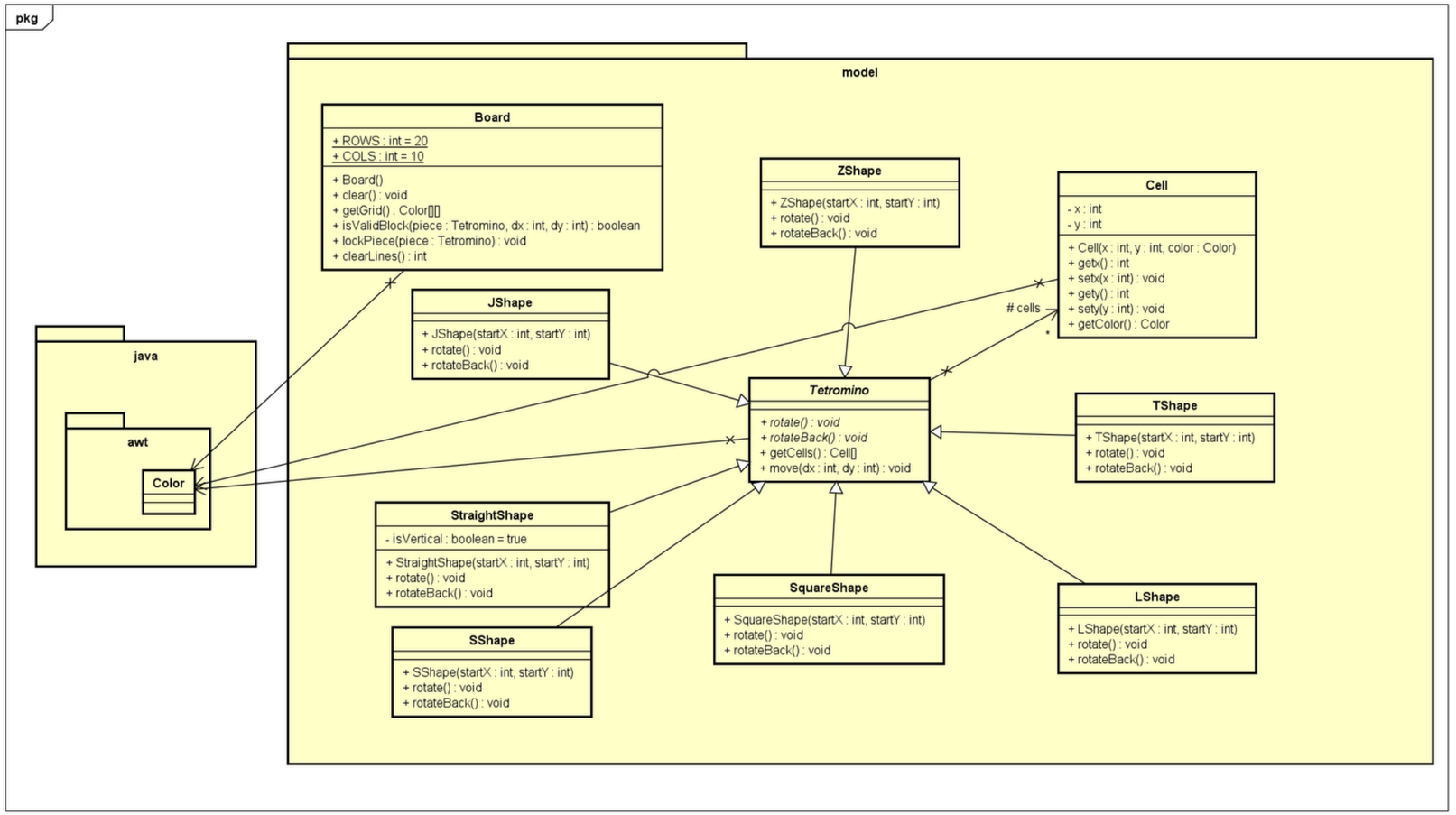
- The GameEngine class runs the main game loop, tracking scores and moving the blocks based on player input.
- A SoundController handle background music and sound effects during gameplay.



7. Model Package class diagram

Internal structure of the game's Model package

- The Board class manages the overall gameplay grid and handles clearing filled lines.
- Various block shapes inherit from a parent Tetromino class, constructed using individual grid Cell objects.



8. OOP technique used

ABSTRACTION

Example of usage:

- Tetromino class is decleared as an abstract class with abstract methods: rotate() and rotateBack() which are later used or Overriding.

```
public abstract class Tetromino {
    protected Cell[] cells = new Cell[4];
    protected Color pieceColor;

    public abstract void rotate();
    public abstract void rotateBack();

    public Cell[] getCells() { return cells; }

    public void move(int dx, int dy) {
        for (Cell cell: cells){
            cell.setx(cell.getx() + dx);
            cell.sety(cell.gety() + dy);
        }
    }
}
```

8. OOP technique used

ENCAPSULATION

Example of usage:

- Private attributes inside Cell class (x, y, color) are only accessible and modifiable by getters and setters (getx(), gety(), getcolor(), ...).

```java
public class Cell {
    private int x;
    private int y;
    private Color color;

    public Cell(int x, int y, Color color) {
        this.x = x;
        this.y = y;
        this.color = color;
    }

    public int getx() { return x; }
    public void setx(int x) { this.x = x; }
    public int gety() { return y; }
    public void sety(int y) { this.y = y; }
    public Color getColor() { return color; }
}
```

8. OOP technique used

INHERITANCE

Example of usage:

- JShape class is made by inheriting from Tetromino class and MenuPanel is made by inheriting from JPanel class.

```
public class JShape extends Tetromino {
    public JShape(int startX, int startY) {
        this.pieceColor = Color.CYAN;

        cells[0] = new Cell(startX + 1, startY, pieceColor);
        cells[1] = new Cell(startX + 1, startY + 1, pieceColor);
        cells[2] = new Cell(startX + 1, startY + 2, pieceColor);
        cells[3] = new Cell(startX, startY + 2, pieceColor);
    }

    @Override
    public void rotate() {...}

    @Override
    public void rotateBack() {...}
}
```

```
public class MenuPanel extends JPanel {
    private BufferedImage bgImage;

    public MenuPanel(JPanel parentContainer, CardLayout layout, GameEngine gameEngine) {
        setLayout(new BoxLayout(this, BoxLayout.Y_AXIS));
        setOpaque(false);
```

8. OOP technique used

POLYMORPHISM

Example of usage:

- currentPiece attributes in GameEngine class is decleared as abstract type Tetromino which holds any one of 7 different shape objects at runtime.
- createNextPiece in GameEngine is a method that can return any of the 7 different shape objects.

```
private Tetromino createNextPiece() {
    int shapeType = random.nextInt( bound: 7);
    if (shapeType == 0) {
        return new StraightShape( startX: Board.COLS / 2,  startY: -1);
    } else if (shapeType == 1) {
        return new SquareShape( startX: Board.COLS / 2,  startY: -1);
    } else if (shapeType == 2) {
        return new LShape( startX: Board.COLS / 2,  startY: -1);
    } else if (shapeType == 3) {
        return new JShape( startX: Board.COLS / 2,  startY: -1);
    } else if (shapeType == 4) {
        return new TShape( startX: Board.COLS / 2,  startY: -1);
    } else if (shapeType == 5) {
        return new SShape( startX: Board.COLS / 2,  startY: -1);
    } else {
        return new ZShape( startX: Board.COLS / 2,  startY: -1);
    }
}
```

8. OOP technique used

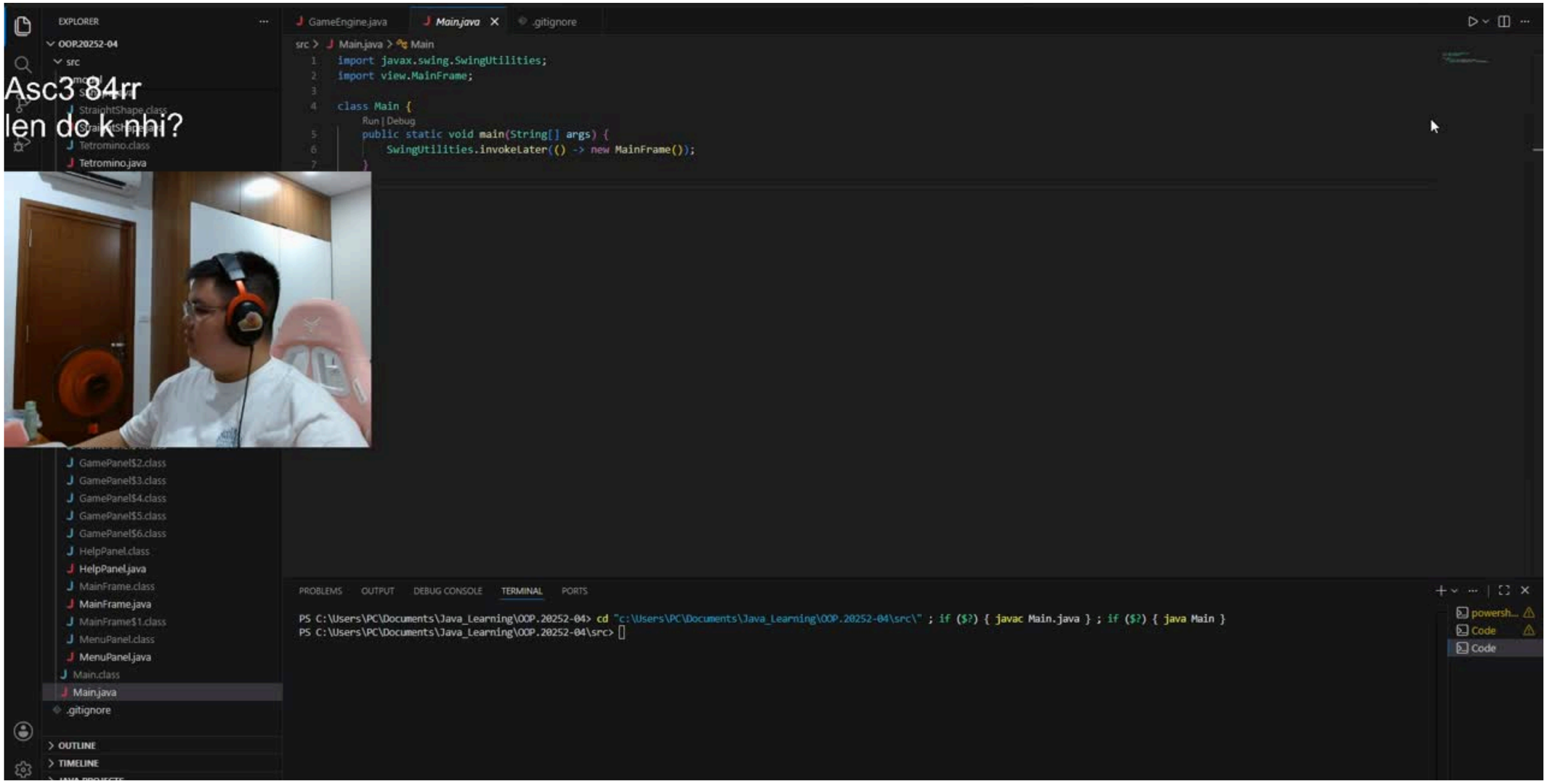
COMPOSITION

Example of usage:

- GameEngine class is built entirely by composing other objects as attributes. It has a "has-a" relationship with every component it needs.

```
public class GameEngine {
    private Board board;
    private Tetromino currentPiece;
    private int score;
    private boolean isGameOver;
    private boolean isPaused;
    private Timer timer;
    private Random random;
    private Runnable viewUpdater;
    private int scoreMultiplier = 1;
    private SoundController soundManager;
```

9. Demo video


Asc3 84rr
len do k nhi?
EXPLORER
OOP.20252-04
src
Tetromino.class
Tetromino.java
GameEngine.java
Main.java
.gitignore
src > Main.java > Main
import javax.swing.SwingUtilities;
import view.MainFrame;
class Main {
Run | Debug
public static void main(String[] args) {
SwingUtilities.invokeLater(() -> new MainFrame());
}
GamePanel$2.class
GamePanel$3.class
GamePanel$4.class
GamePanel$5.class
GamePanel$6.class
HelpPanel.class
HelpPanel.java
MainFrame.class
MainFrame.java
MainFrame$1.class
MenuPanel.class
MenuPanel.java
Main.class
Main.java
.gitignore
OUTLINE
TIMELINE
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\PC\Documents\Java_Learning\OOP.20252-04> cd "c:\Users\PC\Documents\Java_Learning\OOP.20252-04\src\" ; if ($?) { javac Main.java } ; if ($?) { java Main }
PS C:\Users\PC\Documents\Java_Learning\OOP.20252-04\src>
powersh...
Code
Code

HUST

hust.edu.vn fb.com/dhbkhn

THANK YOU !